CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: GuildHall

Prepared By: Leanda, John Luis Catarina

Version: 1.0

Date: 2/18/2026
Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/18/2026	John Luis Leanda	Initial draft	Draft
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY..... 4

1.0 INTRODUCTION..... 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION..... 5

3.0 NON-FUNCTIONAL REQUIREMENTS..... 9

4.0 SYSTEM ARCHITECTURE..... 11

5.0 API CONTRACT & COMMUNICATION..... 12

6.0 DATABASE DESIGN..... 17

7.0 UI/UX DESIGN..... 18

8.0 PLAN..... 21

EXECUTIVE SUMMARY

1.1 Project Overview

GuildHall is a gamified, location-based community service platform designed to facilitate local volunteering and mutual aid through an "Adventurer's Guild" theme. Users can post localized "Quests" (tasks or requests for help) and earn Experience Points (XP) upon completion. The ecosystem consists of a Spring Boot REST API, a MySQL relational database, a React Vite web dashboard for community management, and a Kotlin Android application for on-the-go quest tracking and location services.

1.2 Objectives

1. Develop a functional community service platform with authentication, quest creation and catalog, and quest acceptance.
2. Implement a leveling and XP system to encourage user engagement and local volunteering.
3. Ensure seamless data synchronization between the Android mobile client (for field use) and the React web client (for management).
4. Architect a "Guild" system that filters content based on the user's geographical vicinity.
5. Develop a centralized Spring Boot backend that handles complex business logic, such as quest state transitions and user progression.
6. Implement JWT-based authentication to protect user data and ensure quest integrity across all integrated systems.

1.3 Scope

Included Features:

- User or "Adventurer" profiles featuring levels, XP bars, and "Rank" or "Skill" tags.
- Secure login using JWT and BCrypt hashing. Support for native user registration and Google OAuth integration.
- Quest Board (CRUD): Creating, viewing, bidding on, and completing quests.
- Grouping users and quests into communities or "Guilds" based on specific city or neighborhood boundaries.
- Admin or "Guildmaster" panel for community management

- Real-time status updates.
- Responsive web interface
- Native Android mobile application
- MySQL schema to handle complex relationships between users, guilds, and quest logs.
- Image uploading
- Real-time (Polling or WebSocket) coordination between the Poster and the Adventurer (user).
- Integration with map API
- Payment system for accomplished quests
- Email sending (SMTP)

Excluded Features:

- Logistics or delivery of physical items
- Video chat
- Public global chat

1.0 INTRODUCTION

1.1 Purpose

This document serves as the formal design specification for GuildHall, a gamified community service platform. It outlines the architectural framework, technical requirements, and integration strategies necessary to connect a Spring Boot backend, a React web frontend, and a Kotlin Android mobile application. This document ensures that the "Adventurer Guild" logic and location-based features are consistently implemented across all system tiers.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: GuildHall

Domain: Community Service / Hyper-local Volunteering

Primary Users: Adventurers (Users), Guildmasters (Local Moderators)

Problem Statement: Local communities often lack a centralized, engaging way to request small favors or find volunteering opportunities. Existing social media groups are cluttered and lack accountability or incentive systems.
Solution: A themed "Adventurer's Guild" platform that gamifies real-world help through a Quest/XP system, providing a structured environment for community cooperation with dedicated mobile and web interfaces.

2.2 Core User Journeys

Journey 1: New Adventurer Registration & Onboarding

1. User visits the Kotlin Mobile App or React Web App.
2. User selects "Continue with Google" (Google OAuth Integration).
3. System verifies the Google Token, creates a new record in the `Users` table, and issues a GuildHall JWT.
4. System sends a "Welcome to the Guild" SMTP Email to the user's Gmail address.
5. User views their profile, initialized at Level 1 with 0 XP.

Journey 2: Posting a Quest

1. Poster (User) fills out the "Quest Commission" form.
2. Poster selects a location using a Map Search/Picker. This can be skipped if a location is not needed.
3. System calls the Google Places/Maps API to fetch precise coordinates (Latitude/Longitude) and the formatted address for the quest.
4. Poster enters the "Reward" amount and proceeds to the Stripe Sandbox checkout.
5. System records the payment in the Payments table and sets the Quest status to OPEN. The quest is now pinned to the Global Guild Map.

Journey 3: Completing a Quest

1. Helper (User) arrives at the location provided by the Map API (Only if the quest involves a physical location).
2. Upon completion, the Helper uploads a photo (File Upload Integration) via the app.
3. The Poster receives a notification. They review the photo and the work.

4. The Poster clicks "Confirm Completion."
5. System Logic Triggers:
    - Payment is finalized.
    - XP is automatically awarded to the Helper.
    - SMTP Email is sent to the Helper: "Your quest was accepted by the Poster! +50 XP."

Journey 4: Guildmaster Validation & Reward

1. Guildmaster (Admin) logs into the React Dashboard.
2. They see a global log of all quests to ensure no "illegal" or "scam" quests are posted.
3. Guildmaster has the power to "Flag" or "Delete" quests that violate community standards.
4. Dispute Resolution: If a Helper claims they finished but the Poster refuses to pay, the Guildmaster reviews the Uploaded Proof and can manually trigger the completion.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication & Security: User registration/login with JWT, BCrypt password hashing, and Google OAuth integration.
2. Quest Board (Core CRUD): Full Create, Read, Update, and Delete capabilities for Quests.
3. Map API integration for location selection and distance filtering.
4. File Upload: Uploading image-based "Proof of Work."
5. Payment Gateway.
6. Email (SMTP): Automated welcome and quest notification emails.
7. Role-Based Access Control: Distinct views and API restrictions for Adventurers (Users) and Guildmasters (Admins).
8. Mobile & Web Frontends: Responsive React Vite site and Kotlin Android app.
9.

SHOULD HAVE

1. Quest-Linked Chat: Real-time messaging between Poster and Helper.
2. XP & Leveling System: Backend logic to calculate user rank based on completed quest difficulty.
3. Search & Filtering: Ability to filter quests by category (e.g., Tutoring, Manual Labor) or distance.
4. User Profile Dashboard: A view for users to see their current Level, XP progress, and quest history.

COULD HAVE

1. Quest Leaderboard: A global ranking of top adventurers in a specific Guild location.
2. Achievement Badges: Special icons for completing specific types of quests (e.g., "Scholar" for 5 tutoring quests).
3. Dark Mode: Themed UI toggle for both Web and Mobile.
4. Push Notifications: Sending alerts to the Android app when a nearby quest is posted.

WON'T HAVE

1. Real Money Transactions: Only Sandbox/Test mode will be implemented.
2. Advanced Analytics: Complex data visualization for "Grandmasters" is deferred.
3. Live GPS Tracking: We will show quest pins, but not the live moving dot of the adventurer for privacy/complexity reasons.
4. Multi-language Support: The initial system will be English-only.

2.4 Detailed Feature Specifications

Feature: Authentication & Identity

- **Screens:** Registration, Login, Profile View
- **Fields:** Email, Password, Full Name, Profile Picture (Optional)
- **Validation:** Email format, password complexity (min 8 chars), unique email check.
- **Security: BCrypt** hashing, **JWT** stateless tokens, **Google OAuth 2.0** flow.

- **API Endpoints:**
    - POST /api/auth/register
    - POST /api/auth/login
    - POST /api/auth/google
    - GET /api/auth/me

Feature: The Quest Board (Core CRUD & Maps)

- **Screens:** Map View, Quest List, Post Quest Form
- **Integration: Google Maps API** for coordinate selection and distance-based filtering.
- **Search/Filter:** Filter and Category.
- **API Endpoints:**
    - GET /api/quests (Supports query params: ?lat=...&long=...&radius=10)
    - POST /api/quests (Creates quest with location data)
    - GET /api/quests/{id} (Detailed view with map coordinates)
    - PUT /api/quests/{id}/accept (Links Adventurer to Quest)

Feature: Payments (Bounty System)

- **Screens:** Checkout, Payment Success/Failure
- **Integration: Stripe Sandbox** (Test Mode).
- **Persistence:** Payment results and Transaction IDs stored in the payments table.
- **API Endpoints:**
    - POST /api/payments/create-checkout-session (Initiates Stripe session)
    - GET /api/payments/verify/{session_id} (Backend validation of payment status)

Feature: Completion & Proof (File Upload)

- **Screens:** Upload Proof (Mobile), Review Proof (Web)
- **Functions:** Upload JPG/PNG from Android camera; Gallery view on React.
- **Storage:** Files saved to server directory; paths stored in quests table.
- **API Endpoints:**
    - POST /api/quests/{id}/complete (Multipart request containing image file)
    - GET /api/quests/{id}/proof (Serves the uploaded image to the Poster/Guildmaster)

Feature: Quest Chat (Real-Time)

- **Screens:** Chat Overlay (within Quest Detail)
- **Functions:** Text-based messaging between Poster and Helper.
- **Implementation: WebSocket** (STOMP) for live updates or **Short Polling** (every 5s).
- **API Endpoints:**
    - WS /ws/chat (WebSocket connection point)
    - GET /api/quests/{id}/messages (Fetch message history)

Feature: Guildmaster Panel (Admin)

- **Screens:** Global Quest Logs, User Management, Dispute Resolution
- **Access Control: Role-Based Access Control (RBAC)**; requires ROLE_GUILDMASTER.
- **Functions:** Delete inappropriate quests, ban users, manually trigger XP rewards for disputes.
- **API Endpoints:**
    - GET /api/admin/quests (All quests regardless of status)
    - DELETE /api/admin/quests/{id} (Moderation action)

- PATCH /api/admin/users/{id}/role (Promote users)

2.5 Acceptance Criteria

AC-1: Integrated User Onboarding (Security & Email)

Given I am a new user on the web or mobile app,

When I register using a native form or Google OAuth,

Then the system must hash my password using BCrypt (if native) and save my record in the MySQL database,

And I must receive a "Welcome to the Guild" SMTP Email,

And the system must return a valid JWT to my device for subsequent protected requests.

AC-2: Quest Commissioning (Maps & Payments)

Given I am a logged-in Adventurer (User),

When I create a quest and select a location via the Map API (optional),

And I complete the Stripe Sandbox payment for the bounty,

Then the quest must appear on the global "Guild Board",

And the transaction must be recorded in the payments table with a "Pending" status.

AC-3: Quest Completion & Verification (File Upload)

Given I am an Adventurer (User) who has accepted a quest,

When I coordinate with the poster via the Real-Time Chat,

And I upload an image file as "Proof of Work,"

Then the quest status must update to PENDING_VERIFICATION,

And the image must be stored on the server and accessible via the Guildmaster/poster's dashboard.

AC-4: Reward Distribution & Validation (Logic & RBAC)

Given I am the Adventurer (User) posting the quest (or a Guildmaster resolving a dispute),

When I review the uploaded proof and click "Approve,"

Then the system must update the Helper's XP in the database,

And an SMTP Email notification must be sent to the Helper,

And the quest must transition to the COMPLETED state.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: ≤ 2 seconds for 95% of standard REST requests.
- Map Rendering: Initial map pins must load within 3 seconds of GPS lock.
- File Upload Latency: Proof-of-work images (up to 5MB) must finish processing within 5 seconds on a standard 4G/LTE connection.
- Concurrent Users: Support 100 concurrent adventurers across Web and Mobile.
- Real-time Latency: Chat messages must be delivered within 1 second using WebSockets or Polling.

3.2 Security Requirements

- Encryption: HTTPS/TLS 1.3 for all data in transit.
- JWT-based stateless authentication with a 24-hour expiration.
- Secure handling of Google OAuth 2.0 tokens; internal IDs must be linked to Google sub (subject) IDs.
- Password hashing using BCrypt with 12 salt rounds.
- Implementation of Spring Security filters to prevent SQL Injection and XSS.
- Role-Based Access Control (RBAC): Server-side verification of roles (ROLE_ADVENTURER, ROLE_GUILDMASTER) before accessing restricted endpoints.

3.3 Compatibility Requirements

- Web Frontend (React Vite): Chrome, Firefox, Safari, and Edge. Fully responsive design from 320px (small mobile) to 2560px (4K monitors).
- Mobile Frontend (Kotlin): Android API Level 26+ (Android 8.0 Oreo and above) to support modern notification and location services.

3.4 Usability Requirements

- New users should be able to complete "Continue with Google" and find their first local quest within 3 minutes.
- WCAG 2.1 Level AA compliance.
- Mobile-First Design: Minimum 48x48px touch targets for quest "Accept" and "Complete" buttons.
- Error Handling: Meaningful error messages (e.g., "Payment Failed: Insufficient Funds" instead of "Error 500") with recovery paths.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram

```
┌──────────────────────┐   ┌──────────────────────┐
│  WEB CLIENT   │   │ MOBILE CLIENT │
│   (React)     │   │  (Android)    │
│ • TypeScript  │   │  • Kotlin     │
│ • Tailwind CSS│   │  • Compose    │
└──────────┬───────────┘   └──────────┬───────────┘
```

```
│              │
──────────┬──────────────────────┘
          │
  ┌──────────────▼──────────────┐
  │  BACKEND API      │
  │  (Spring Boot)    │
  │  • Java 17        │
  │  • Spring Security │
  │  • JPA/Hibernate  │
──────────┬──────────────────────┘
          │
  ┌──────────────▼──────────────┐
  │  DATABASE         │
  │  (PostgreSQL)     │
  │  • Users table    │
  │  • Products table │
  │  • Orders table   │
  │  • Cart items table│
──────────────────────────────┘
```

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.marketplace.com/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

```
json
{
  "success": boolean,
  "data": object|null,
  "error": {
    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Product Endpoints

GET /products

Query: ?page=1&limit=20&search=keyword&category=electronics

Response: {products: [...], pagination: {page, limit, total, pages}}

GET /products/{id}

Response: {product: {id, name, description, price, stock, imageUrl, category}}

POST /products (Admin only)

Body: {name, description, price, stock, imageUrl, category}

Response: {product: {...}}

PUT /products/{id} (Admin only)

Body: {name?, description?, price?, stock?, imageUrl?, category?}

Response: {product: {...}}

Cart Endpoints

GET /cart (Authenticated)

Response: {cart: {id, items: [...], total, itemCount}}

POST /cart/items

Body: {productId, quantity}

Response: {message: "Added to cart", cartItem: {...}}

PUT /cart/items/{itemId}

Body: {quantity}

Response: {message: "Cart updated", cartItem: {...}}

DELETE /cart/items/{itemId}

Response: {message: "Removed from cart"}

Order Endpoints

POST /orders

Body: {shippingAddress: {fullName, address, city, zipCode, country}}

Response: {order: {id, orderNumber, total, status, items: [...], createdAt}}

GET /orders

Response: {orders: [...]}

GET /orders/{id}

Response: {order: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist

- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

```
json
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2024-01-28T10:30:00Z"
}

{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2024-01-28T10:30:00Z"
```

}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD

```
USERS (1) ——— (1) CARTS (1) ——— (*) CART_ITEMS (*) ——— (1) PRODUCTS
   |                    |
   |                    |
  (*)                  (*)
   |                    |
   |                    |
ORDERS (1) ——— (*) ORDER_ITEMS (*) ———┘
```

Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)

- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[🏠 Home] [🔍 Search] [🛒 Cart] [👤 Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements